Rapport d'Implémentation : Environnement de Simulation de Drone pour l'Apprentissage par Renforcement

Sommaire

Introduction....1
1. Rôle de l'Environnement en Apprentissage par Renforcement....1
2. Modélisation du Système (États, Actions et Transitions)....2
    2.1 Espace des États....2
    2.2 Espace des Actions....2
    2.3 Dynamique des Transitions et Non-Déterminisme....3
3. Choix d'Implémentation et Architecture....3
    3.1 Architecture Orientée Objet....3
    3.2 Adaptation pour l'Automatisation....3
Conclusion....4

Introduction

Ce rapport détaille la conception et l'implémentation d'un environnement de simulation modélisant un drone de patrouille autonome. L'objectif principal est de fournir un cadre formel et fonctionnel permettant, à terme, l'entraînement d'agents via des algorithmes d'Apprentissage par Renforcement (RL). Le système exige de l'agent qu'il maximise la couverture d'une zone divisée en secteurs tout en gérant une batterie limitée et en évitant des zones de danger.

1. Rôle de l'Environnement en Apprentissage par Renforcement

Dans le paradigme de l'Apprentissage par Renforcement, l'environnement est l'entité avec laquelle l'agent interagit en modélisant un Processus de Décision Markovien (MDP). Notre environnement joue trois rôles cruciaux :

Observation : Il expose un état discret et fini à l'agent à chaque instant t.

Application des actions : Il valide et exécute les commandes de l'agent selon des préconditions strictes (ex: un drone crashé ne peut pas voler).

Dynamique de transition : Il calcule l'état à t+1 (positions, batterie, statut) de manière déterministe ou stochastique, et gère la physique du monde (zones de danger, retour à la base).

Note sur la récompense : Comme précisé dans le manuel développeur, l'environnement simule la physique pure. La fonction de récompense sera encapsulée dans une surcouche propre à l'agent RL.

2. Modélisation du Système (États, Actions et Transitions)

2.1 Espace des États

L'état du système encapsule toutes les informations nécessaires pour prendre une décision. Dans notre implémentation (classe Etat en Python), il est caractérisé par :

La position : Des coordonnées entières (posX,posY) bornées par maxX et maxY.

Les ressources : Le niveau de batterie, un entier défini dans l'intervalle [0,100].

Le statut opérationnel : Trois variables booléennes garantissant la cohérence physique : airborne (en vol), docked (amarré), et crashed (détruit).

La perception : obstacle_distance, indicateur abstrait informant l'agent de l'imminence d'un danger.

La couverture : visZones, un ensemble retenant les identifiants des zones explorées pour évaluer la performance de patrouille.

Le terrain est mathématiquement divisé en 4 zones, selon la fonction zoneOf(x,y) séparant la grille en quatre.

2.2 Espace des Actions

L'agent dispose d'un ensemble d'actions discrètes, chacune soumise à des préconditions (invariants d'état) :

Mouvements standards (move_step, return_home) : Coûtent 1 unité de batterie. Le déplacement y est contraint par les bords de la grille et modifie la position courante.

Mouvement défensif (avoid_maneuver) : Coûte 2 unités de batterie. Il illustre le compromis entre sécurité et énergie en cherchant à garantir (si possible) une transition vers une case n'appartenant pas à l'ensemble Danger.

Gestion du cycle de vie (takeoff_cmd, dock_cmd, undock_cmd, charge_step) : Permettent de gérer les transitions entre le vol, l'amarrage à la base (0,0) et le rechargement par paliers de 5%.

État terminal (emergency_stop) : Force le crash du drone.

2.3 Dynamique des Transitions et Non-Déterminisme

La spécification formelle utilise la clause ANY pour désigner des choix non déterministes parmi les cellules voisines. Lors de l'implémentation en Python, ce non-déterminisme a été modélisé pour simuler un mouvement général sans cap explicite (par exemple via la fonction random.choice sur les cellules voisines valides).

Toute transition qui mène les coordonnées (posX,posY) dans l'ensemble Danger met immédiatement à jour la variable obstacle_distance à 0. Bien que cela n'entraîne pas un crash automatique dans la fonction de mouvement pure (comme soulevé dans les limitations du manuel développeur), cela signale à la future surcouche RL qu'un état critique (terminal) a été atteint.

3. Choix d'Implémentation et Architecture

3.1 Architecture Orientée Objet

Le code Python (Zone.py) a été conçu de manière modulaire :

La classe Couple : Implémente __eq__ et __hash__ pour manipuler aisément les coordonnées (x,y) et s'intégrer facilement dans des structures de données comme les ensembles ou les dictionnaires.

La classe Etat : Sépare proprement l'état actuel de la logique de calcul. Elle rend le code compatible avec la mémorisation requise par les algorithmes RL.

Les classes Drone et Environnement : Maintiennent les invariants (ex: un drone amarré voit ses coordonnées fixées à la base).

3.2 Adaptation pour l'Automatisation

Comme documenté dans l'étude de cas, le code initial intégrait une boucle while interactive basée sur des input() utilisateur. Pour rendre l'environnement compatible avec un apprentissage automatisé :

1. **Suppression des E/S bloquantes :** La boucle interactive a été dissociée de l'environnement de base. L'environnement s'instancie désormais de manière programmatique via un script RL maître.

2. **Fonction de transition (Model-Based) :** Une méthode env.successeurs(etat) a été implémentée pour permettre aux algorithmes de planification (ex: graphes d'exploration) de calculer tous les états suivants possibles.

Conclusion

L'environnement développé offre une représentation discrète, robuste et formellement vérifiée de la mission du drone. En respectant les préconditions de chaque opération et en encapsulant proprement la dynamique physique, il fournit une base saine pour l'intégration f uture d'un agent d'Apprentissage par Renforcement chargé de maximiser la découverte des 4 zones sous contrainte énergétique.